

API Connector

pour Odoo 18

Appels sortants REST, JSON-RPC, GraphQL et SOAP depuis Odoo

Version 18.0.1.2.0

IROKOO

sbt@irokoo.fr | https://www.irokoo.fr

Historique des versions

Version	Date	Objet
18.0.1.0.0	10/2026	Documentation en français et en anglais, captures d'écran de la version 18
18.0.1.2.0	10/2026	Variables en lignes, détectées et créées à l'enregistrement, appliquées aussi aux en-têtes ; corps construit par lignes ; variables lues sur un champ Odoo ; aperçu de la requête

Sommaire

1 Présentation 4
1.1 À quoi sert le module 4
1.2 Fonctions 4
1.3 Fiche du module 4
1.4 Menus 5
2 Installation 5
2.1 Prérequis 5
2.2 Installer le module 5
2.3 Ce que l'installation ajoute 5
2.4 Mettre à jour depuis la version 18.0.1.0.0 5
3 Prise en main : un premier appel 5
4 Collections 6
4.1 Paramètres de collection 7
4.2 En-têtes et variables 8
5 Points de terminaison 8
5.1 Configuration de base 8
5.2 En-têtes 9
5.3 Corps de la requête 10
5.4 Réponse et statistiques 11
6 Authentification 12
6.1 Hériter ou définir 12
6.2 Méthodes 12
6.3 OAuth 2.0 13
7 Variables 13
7.1 Onglet Variables 13
7.2 Valeur d'une variable 14
7.3 Enregistrement de test 15
8 Aperçu de la requête 15
9 Exécution automatique 15
9.1 Depuis une action planifiée 15
9.2 Lire le résultat 16
10 Surveillance 17
10.1 Journal des appels 17
10.2 Conservation 18
11 Export et import 18
11.1 Exporter 18
11.2 Importer 18

11.3 Collections Postman 19
12 Droits d'accès 19
13 Référence technique 19
13.1 Modèles 19
13.2 Méthodes utiles 20
13.3 Variables en JSON 20
13.4 Supprimer un point de terminaison critique 20

1 Présentation

1.1 À quoi sert le module

API Connector permet à Odoo d'appeler des API externes. Chaque appel se décrit dans un formulaire, à la manière de Postman : adresse, méthode HTTP, authentification, en-têtes, corps de la requête. On l'exécute d'un clic pour le tester, puis on le confie à une action planifiée ou à une action serveur pour qu'il tourne seul. Chaque appel laisse une trace dans un journal.

Le module est le pendant sortant d'Advanced JSON-RPC API, qui expose les données d'Odoo aux appels entrants.

1.2 Fonctions

Fonction	Ce qu'elle fait
Collections	Regroupent les points de terminaison d'une même API et partagent l'URL de base, l'authentification, les en-têtes et les variables.
Points de terminaison	Un appel configuré : type d'API, méthode, chemin, en-têtes, corps, délai, vérification SSL.
Types d'API	REST, JSON-RPC, GraphQL, SOAP/XML.
Authentification	Aucune, authentification de base, jeton Bearer, clé API dans un en-tête, OAuth 2.0 (client credentials).
Variables	Valeurs `{nom}` remplacées dans l'URL, les en-têtes et le corps. Saisies sur la collection ou le point de terminaison, ou lues sur un champ de l'enregistrement envoyé.
Corps construit	Corps JSON ou de formulaire assemblé à partir de lignes clé / variable, avec un type par valeur.
Aperçu	Méthode, URL, en-têtes et corps tels qu'ils partiraient, avant tout envoi.
Exécution	Bouton dans le formulaire ou dans la liste, et appel par code depuis une action planifiée ou une action serveur.
Surveillance	Journal de chaque appel (statut, durée, données envoyées et reçues), taux de succès par point de terminaison et par collection.
Export et import	Configuration au format JSON, et import de collections Postman v2.0 à v2.2.

1.3 Fiche du module

Propriété	Valeur
Nom technique	`iro_api_connector`
Version	18.0.1.2.0
Version d'Odoo	18
Licence	OPL-1
Auteur	IROKOO
Modules Odoo requis	`base`, `web`
Bibliothèques Python	`requests`, `jsonschema`

1.4 Menus

Le module ajoute l'application **Connecteur API** :

Menu	Entrée	Contenu
Configuration	Collections	Les collections et leurs réglages partagés
	Points de terminaison	Tous les appels configurés
	Importer la configuration	Import d'un export JSON ou d'une collection Postman
Surveillance	Journaux d'appel	L'historique des appels, filtré par défaut sur les 7 derniers jours

2 Installation

2.1 Prérequis

Les bibliothèques Python `requests` et `jsonschema` doivent être présentes dans l'environnement d'Odoo. Elles figurent dans le fichier `requirements.txt` du module :

```
pip install -r iro_api_connector/requirements.txt
```

2.2 Installer le module

1. Copier le dossier `iro_api_connector` dans un répertoire du chemin des modules (`addons_path`).
2. Activer le mode développeur, puis **Applications > Mettre à jour la liste des applications**.
3. Rechercher **API Connector** et cliquer sur **Activer**.

2.3 Ce que l'installation ajoute

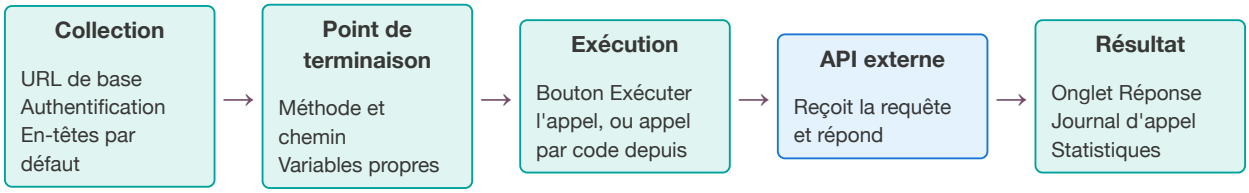
- L'application **Connecteur API** et ses menus.
- Des collections et points de terminaison d'exemple appelant des API publiques (JSONPlaceholder, httpbin.org, api-adresse.data.gouv.fr, Rick and Morty GraphQL). Ils servent de modèles et peuvent être archivés.
- Une action planifiée quotidienne, **API Connector: Cleanup Old Logs**, qui supprime les entrées du journal de plus de 30 jours.

2.4 Mettre à jour depuis la version 18.0.1.0.0

La mise à jour du module convertit les variables saisies en JSON sur les collections et les points de terminaison en lignes de variables. Les adresses appelées ne changent pas. Les champs JSON restent lisibles et modifiables par code (voir Référence technique).

3 Prise en main : un premier appel

Le schéma suivant résume ce qui compose un appel.



Variables de collection

En-têtes personnalisés
Authentification propre ou héritée
Corps de la requête

une action planifiée

Les étapes ci-dessous créent une collection vers httpbin.org, un service public qui renvoie ce qu'il reçoit : la réponse montre exactement les en-têtes et le corps envoyés par Odoo.

1. **Connecteur API > Configuration > Collections > Nouveau**. Nom : `Partner API (demo)`, URL de base : `https://httpbin.org`.
2. Onglet **Authentification** : **Jeton Bearer**, jeton `demo-token-2026`.
3. Onglet **En-têtes et variables** : en-têtes par défaut `{"Accept": "application/json"}`, puis une variable de collection `api_version` de valeur `v2`. Enregistrer.
4. Bouton **Ajouter un point de terminaison**. Nom : `Get contact`, méthode `GET`, chemin `/anything/{api_version}/contacts/{contact_id}`. Enregistrer : la variable `contact_id` est créée, vide, dans l'onglet **Variables**. Lui donner la valeur `42` et enregistrer. L'URL complète affichée sous le nom devient `https://httpbin.org/anything/v2/contacts/42`.
5. Cliquer sur **Exécuter l'appel**, puis ouvrir l'onglet **Réponse**.

L'appel figure aussi dans **Surveillance > Journaux d'appel**, et les compteurs de l'onglet **Statistiques** ont augmenté d'une unité.

4 Collections

Une collection regroupe les points de terminaison d'une même API. Ses réglages servent de valeurs par défaut à tous ses points de terminaison.

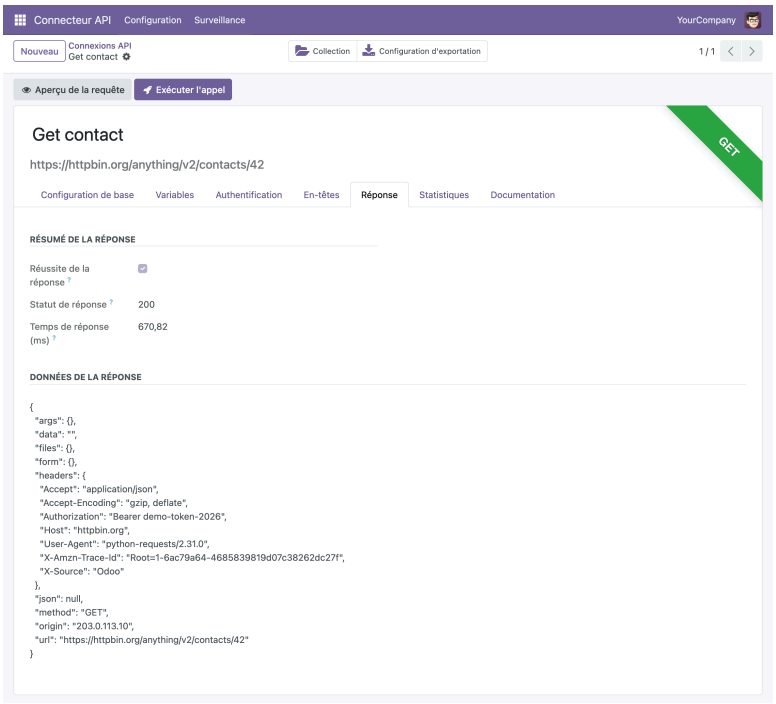


Fig. 1. – Réponse de httpbin.org : l'en-tête `Authorization` vient de la collection, l'URL a reçu ses deux variables.

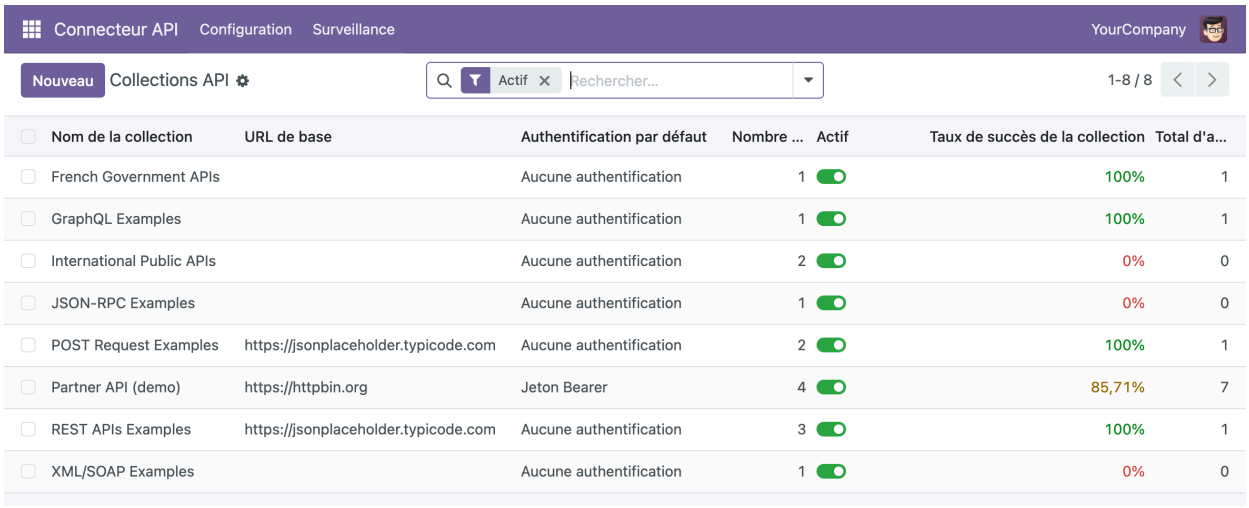


Fig. 2. – Liste des collections : URL de base, authentification par défaut, nombre de points de terminaison, taux de succès et total d'appels.

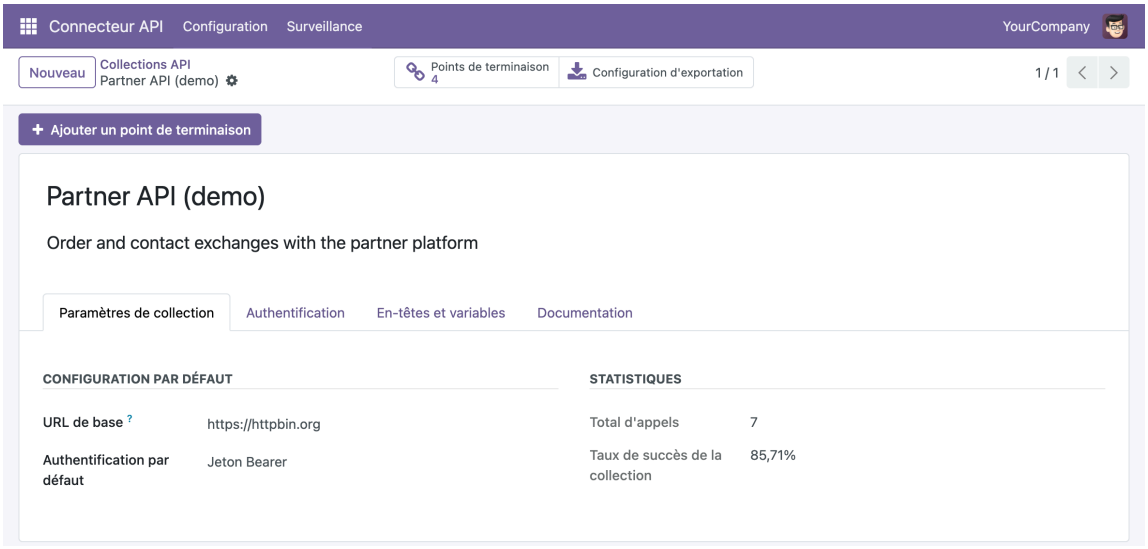


Fig. 3. – Formulaire d'une collection, onglet Paramètres de collection.

4.1 Paramètres de collection

Champ	Rôle
Nom de la collection	Obligatoire.
Description	Texte libre sous le nom.
URL de base	Début commun des adresses, par exemple `https://api.exemple.com`. Utilisée par les points de terminaison qui cochent **Utiliser l'URL de la collection**.
Authentification par défaut	Utilisée par les points de terminaison dont l'authentification est **Hériter de la collection**. Détail au chapitre Authentification.
Total d'appels, taux de succès	Somme des compteurs des points de terminaison de la collection.

Boutons du formulaire :

- **Ajouter un point de terminaison** ouvre un point de terminaison pré-rempli avec l'URL de base, l'authentification et les en-têtes par défaut de la collection.
- **Points de terminaison** affiche la liste des points de terminaison de la collection.

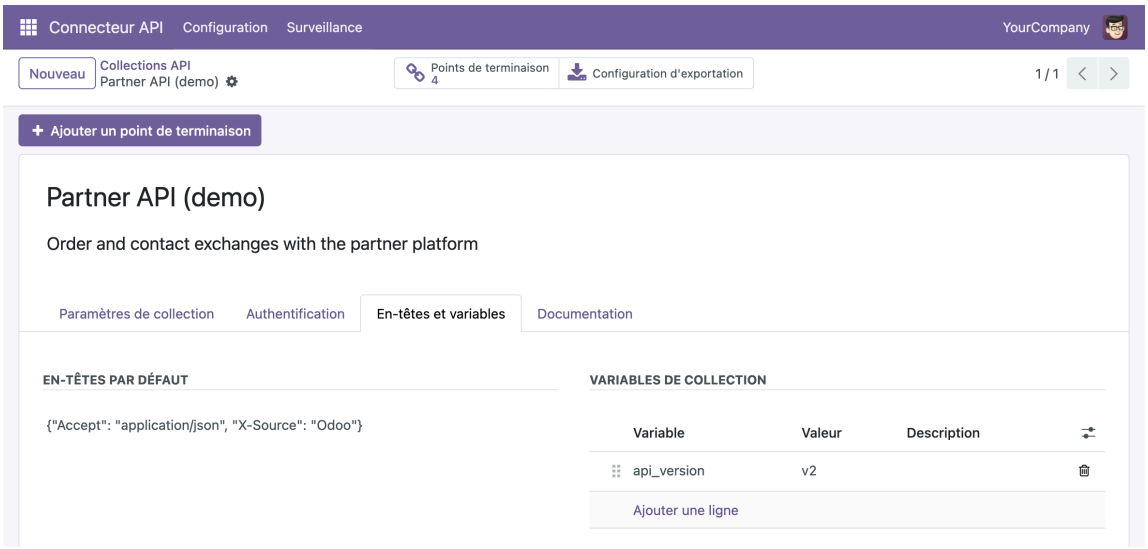


Fig. 4. – Onglet En-têtes et variables : en-têtes par défaut en JSON, variables en lignes.

- **Configuration d'exportation** télécharge la collection et ses points de terminaison au format JSON (voir le chapitre Export et import).

Archiver une collection archive tous ses points de terminaison. La désarchiver les réactive.

4.2 En-têtes et variables

- **En-têtes par défaut** : objet JSON envoyé avec chaque appel des points de terminaison de la collection. Il doit être du JSON valide : l'enregistrement est refusé sinon. Il peut contenir des marqueurs `{nom}`.
- **Variables de collection** : une ligne par variable (nom, valeur, description), partagée par tous les points de terminaison de la collection.

À l'enregistrement de la collection, les marqueurs `{nom}` de son URL de base et de ses en-têtes par défaut qui ne correspondent à aucune variable sont créés, vides, dans cette liste.

5 Points de terminaison

Un point de terminaison décrit un appel. La liste les montre tous, avec la méthode en couleur, le taux de succès et un bouton **Exécuter** par ligne.

5.1 Configuration de base

Champ	Rôle
Collection	Collection de rattachement, facultative.

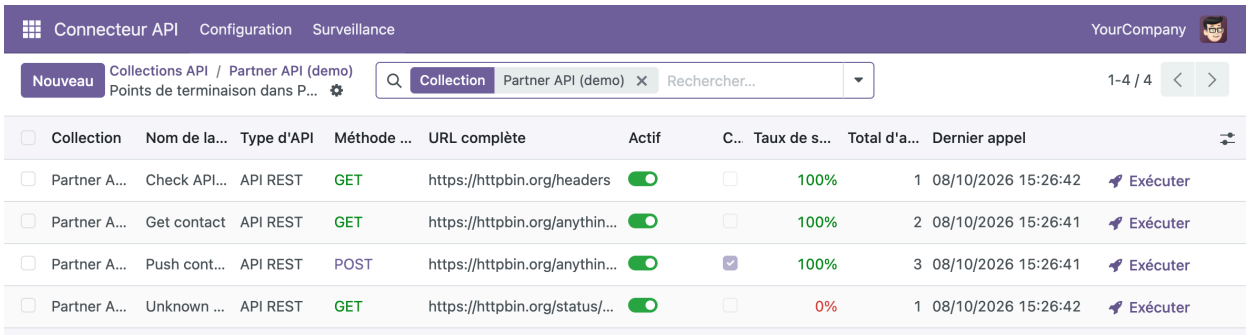


Fig. 5. – Points de terminaison d'une collection. La colonne C. (critique) signale ceux protégés contre la suppression.

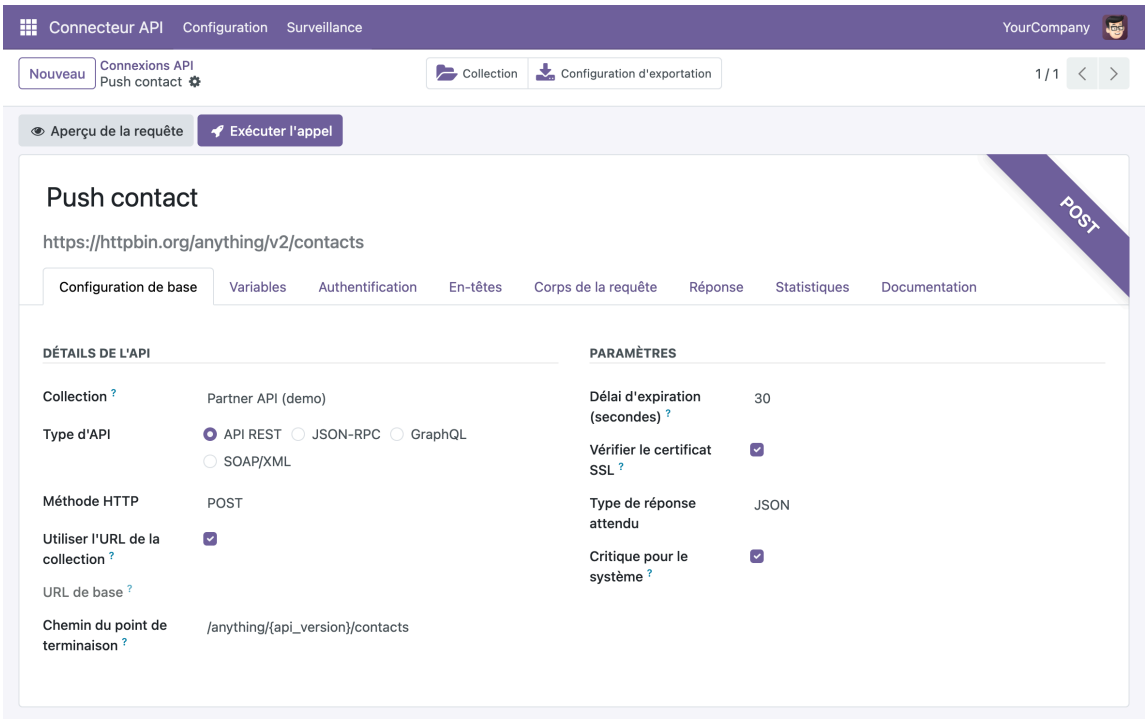


Fig. 6. – Onglet Configuration de base. Le bandeau indique la méthode, l'URL complète est calculée sous le nom.

Type d'API	REST, JSON-RPC, GraphQL ou SOAP/XML. Pour JSON-RPC, GraphQL et SOAP/XML, la méthode passe à POST et n'est plus modifiable. JSON-RPC règle aussi le corps sur JSON.
Méthode HTTP	GET, POST, PUT, PATCH ou DELETE, modifiable en REST.
Utiliser l'URL de la collection	Affiché quand une collection est choisie. Coché, l'URL de base vient de la collection. Décoché, on saisit l'URL de base du point de terminaison.
URL de base	Doit commencer par `http://` ou `https://`.
Chemin du point de terminaison	Ajouté à l'URL de base. Peut contenir des marqueurs `{nom}` (voir le chapitre Variables).
Délai d'expiration (secondes)	De 1 à 300, 30 par défaut.
Vérifier le certificat SSL	Coché par défaut.
Type de réponse attendu	JSON, XML, texte brut ou binaire. Détermine la lecture de la réponse (voir Réponse).
System Critical	Interdit la suppression du point de terminaison. Il reste désactivable avec le champ Actif.

L'**URL complète** est l'URL de base suivie du chemin, après remplacement des variables qui ont une valeur. Elle est recalculée à chaque modification. À l'exécution, l'adresse est calculée à nouveau avec les données passées à l'appel et l'enregistrement envoyé.

5.2 En-têtes

L'onglet **En-têtes** reçoit un objet JSON d'en-têtes personnalisés. À l'envoi, Odoo assemble les en-têtes dans cet ordre, chaque étape l'emportant sur la précédente :

1. en-têtes par défaut de la collection ;
2. en-têtes personnalisés du point de terminaison ;
3. remplacement des marqueurs `{nom}` dans les valeurs ;
4. en-tête d'authentification (`Authorization` ou en-tête de clé API) ;

5. `Content-Type` déduit du type de corps, seulement s'il n'a pas été fourni : `application/json`, `application/xml`, `application/x-www-form-urlencoded` ou `application/octet-stream`.

5.3 Corps de la requête

L'onglet n'apparaît pas pour un appel REST en GET ou DELETE. Pour un corps JSON ou de formulaire, **Mode du corps** choisit entre **Saisi**, le texte tapé, et **Construit par lignes**.

5.3.1 Corps saisi

Type de corps	Envoi
Aucun corps	Pas de corps.
JSON	Le texte saisi, vérifié comme JSON valide au moment de l'appel.
Données de formulaire (urlencoded)	Les paires clé/valeur du champ **Données de formulaire**, saisi en objet JSON.
Données de formulaire (multipart)	Les paires clé/valeur du champ **Données de formulaire**, encodées en `x-www-form-urlencoded` comme le type précédent.
XML, texte brut	Le texte saisi, tel quel.
Binaire (base64)	Le contenu saisi en base64, décodé avant l'envoi.

Les marqueurs `{nom}` du corps et des données de formulaire sont remplacés par les variables (voir le chapitre Variables).

5.3.2 Corps construit par lignes

Colonne	Rôle
Clé	Nom de la valeur dans le corps. En JSON, une clé avec des points crée des objets imbriqués : `customer.address.city` donne `{"customer": {"address": {"city": ...}}}`. En données de formulaire, la clé reste telle quelle.
Source	**Variable** (valeur de la variable nommée) ou **Constante** (valeur saisie).
Type	**Tel quel** garde la valeur reçue, **Texte**, **Entier**, **Décimal** ou **Booléen** la convertissent (`oui`, `true`, `1` pour vrai).
Obligatoire	Une valeur vide bloque l'envoi. L'erreur nomme la clé.

Une valeur vide non obligatoire donne `null` en JSON et est omise en données de formulaire. **Modèle du corps** affiche le corps avec les `{variables}` à la place des valeurs, **Corps pour l'enregistrement de test** le corps réel (voir Enregistrement de test).

5.3.3 Ajouter des champs Odoo

Le bouton **Ajouter des champs Odoo** demande un modèle source (onglet **Variables**). Il ouvre l'arbre des champs de ce modèle, parcouru comme celui de l'export standard : un clic déplie une relation, jusqu'à trois niveaux, **+** ou un double-clic ajoute le champ.

Ajouter au corps crée, pour chaque champ, une variable de route qui lit ce champ (`country_id.code` donne `{country_id_code}`) et une ligne du corps qui l'utilise, de type **Tel quel** : un many2one donne son identifiant, un one2many ou un many2many la liste des identifiants (ou des valeurs quand le chemin continue, par exemple `order_line.product_id.default_code`), une date le format ISO, un champ vide `null`. Le corps passe en mode **Construit par lignes**.

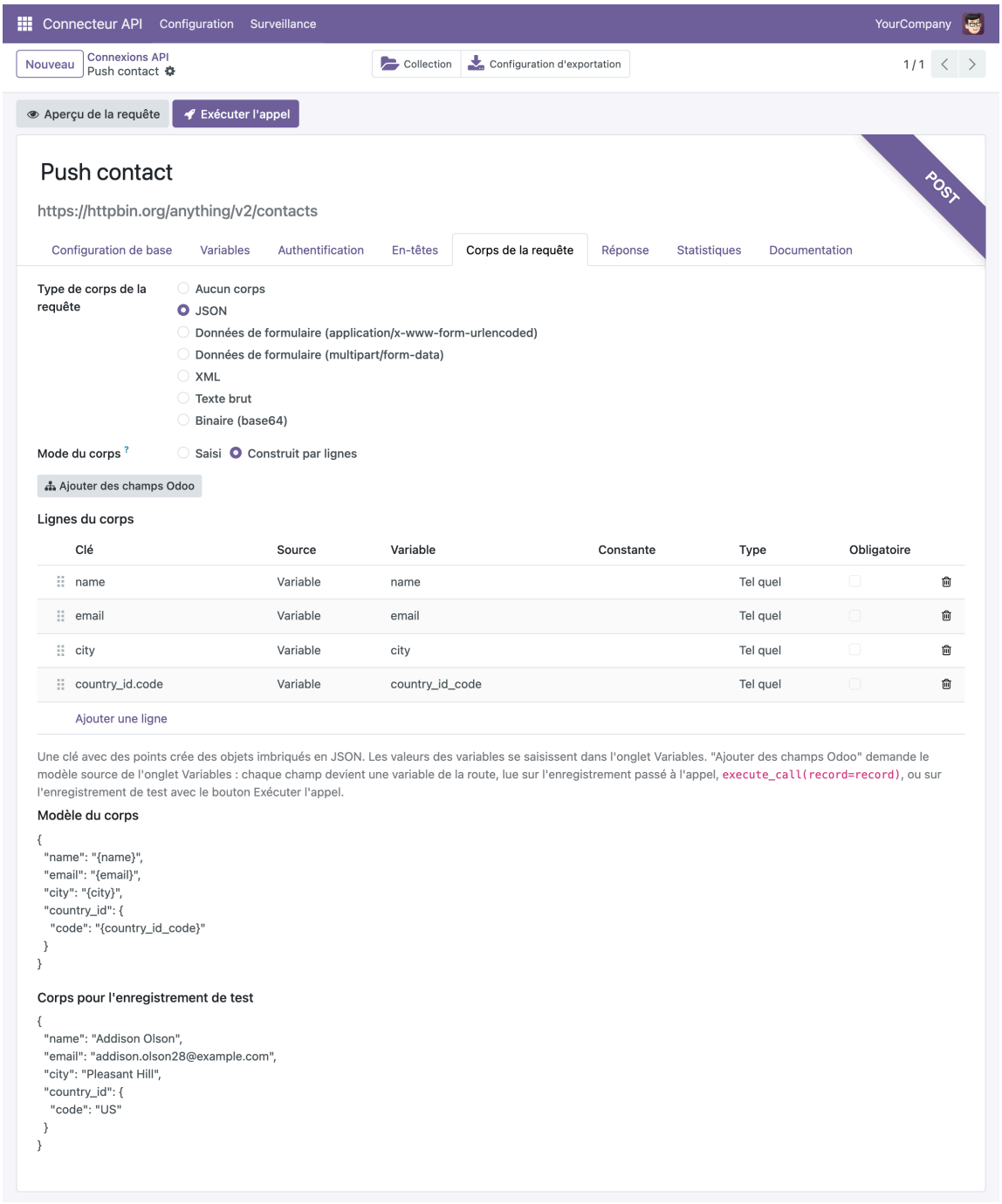


Fig. 7. – Corps construit par lignes, avec son modèle et le corps réel pour l'enregistrement de test.

5.4 Réponse et statistiques

L'onglet **Réponse** affiche le résultat du dernier appel : réussite, statut HTTP, temps de réponse en millisecondes et données reçues. Une réponse JSON est présentée indentée. Avec le type de réponse attendu **Binaire**, les données sont conservées en base64.

Un appel est réussi quand le statut HTTP est inférieur à 400. Un statut 4xx ou 5xx, un délai dépassé ou une erreur de connexion comptent comme un échec. En cas d'erreur de connexion ou de délai dépassé, aucune donnée n'est reçue : le message d'erreur se lit dans le journal, onglet **Détails de l'erreur**.

L'onglet **Statistiques** indique le total d'appels, les appels réussis, le taux de succès et la date du dernier appel.

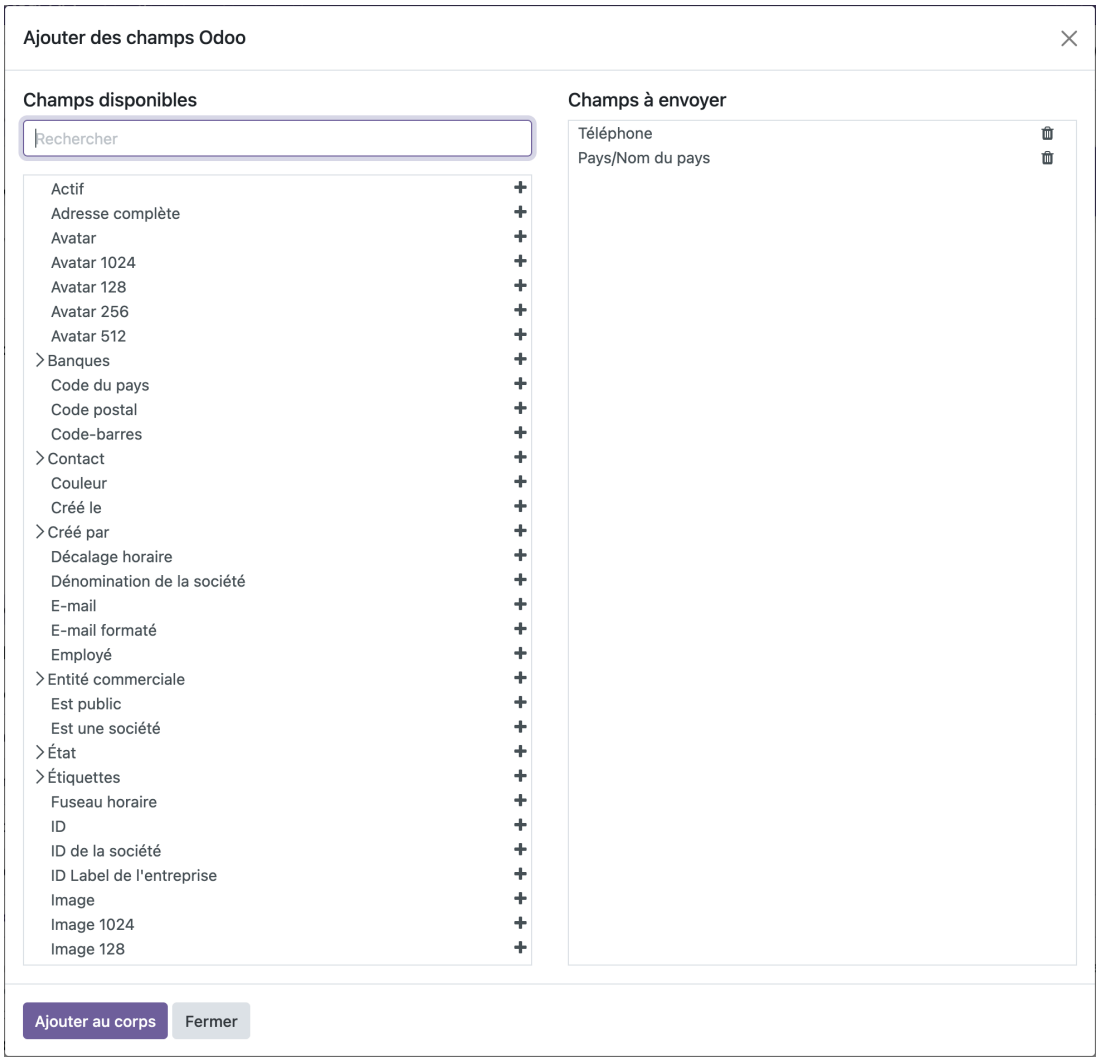


Fig. 8. – Choix des champs à envoyer. Les relations se déplient comme dans l'export standard.

6 Authentification

6.1 Hériter ou définir

Un point de terminaison a l'authentification **Hériter de la collection** par défaut : il utilise alors celle de sa collection. Il peut aussi définir la sienne, qui prime.

6.2 Méthodes

Méthode	Champs	Envoi
Aucune authentification		Rien.
Authentification de base	Nom d'utilisateur, mot de passe	Authentification HTTP Basic.
Jeton Bearer	Jeton Bearer / Clé API	`Authorization: Bearer <jeton>`
Clé API	Nom de l'en-tête Clé API (`X-API-Key` par défaut), Jeton Bearer / Clé API	`<nom de l'en-tête>: <clé>`
OAuth 2.0	ID client, secret client, URL du jeton, portée	`Authorization: Bearer <jeton d'accès>`

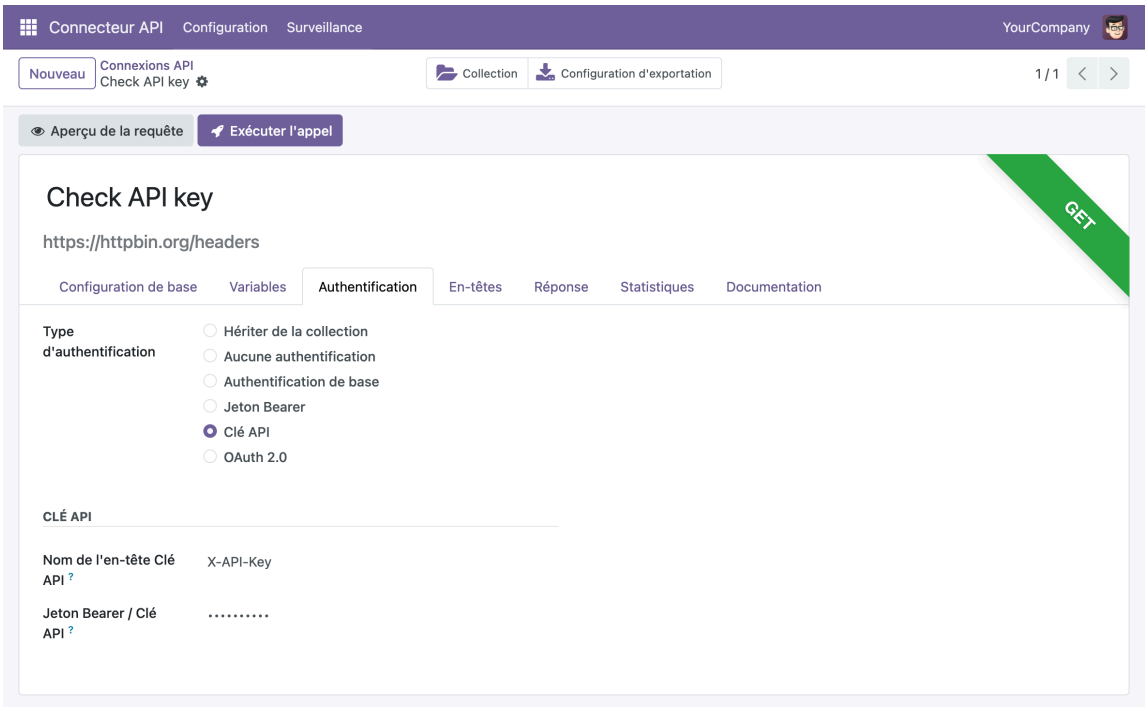


Fig. 9. – Un point de terminaison qui remplace le jeton Bearer de sa collection par une clé API.

Les champs de mot de passe, de jeton et de secret sont masqués à l'écran.

6.3 OAuth 2.0

Le bouton **Obtenir un nouveau jeton d'accès** demande un jeton à l'URL du jeton selon le flux **client credentials** (`grant_type=client_credentials`, avec l'ID client, le secret et la portée). Le jeton d'accès, le jeton de rafraîchissement et la date d'expiration sont enregistrés et une notification affiche le résultat.

- Sur une collection, le jeton obtenu sert aussi aux points de terminaison qui héritent de l'authentification.
- Le module ne renouvelle pas le jeton à son expiration : on clique à nouveau sur le bouton, ou on colle un jeton dans le champ prévu sous les jetons.

7 Variables

Un marqueur s'écrit `{nom}` (lettres, chiffres, _, . et -, en commençant par une lettre ou _). Il est remplacé dans l'URL, les en-têtes et le corps de la requête.

7.1 Onglet Variables

Bloc	Contenu
Enregistrement Odoo	**Modèle source** : modèle des enregistrements envoyés, dont les champs peuvent alimenter les variables. **Enregistrement de test** : un enregistrement de ce modèle, pour les essais.
Variables détectées	Chaque `{nom}` trouvé, où il est utilisé, où il est défini et la valeur qui serait envoyée. Une ligne surlignée n'a pas de valeur.
Variables de la route	Nom, **Champ Odoo** (choisi avec le sélecteur de champ standard), valeur et description.
Variables de la collection	Rappel, en lecture seule, des variables de la collection.

À l'enregistrement d'un point de terminaison, les marqueurs `{nom}` de son URL, de ses en-têtes et de son corps qui ne sont définis ni sur lui ni sur sa collection sont créés, vides, dans **Variables de la route**.

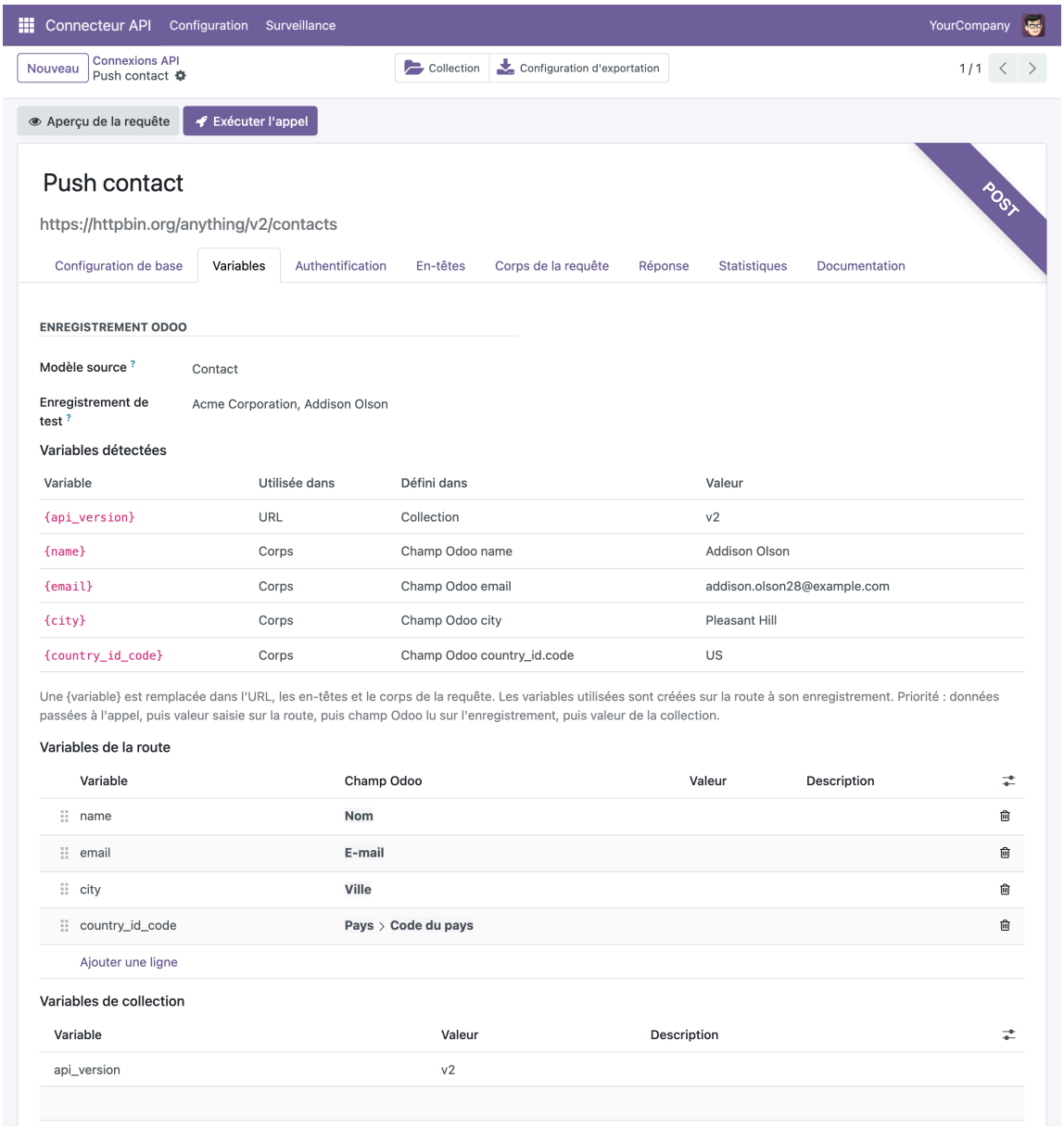


Fig. 10. – Onglet Variables : modèle source, variables détectées avec leur valeur, variables de la route.

7.2 Valeur d'une variable

Rang	Source
1	Données passées à l'appel : `execute_call({'nom': valeur})`.
2	Valeur saisie sur la variable de la route.
3	Champ Odoo de la variable de la route, lu sur l'enregistrement passé à l'appel.
4	Valeur de la variable de la collection.

La première source qui donne une valeur l'emporte. Une variable sans valeur laisse son `{nom}` tel quel dans la requête.

```
connector.execute_call({'contact_id': '42'})   # valeur passée à l'appel
connector.execute_call(record=partner)         # champs lus sur le contact
```

7.3 Enregistrement de test

Les variables qui lisent un champ Odoo ont besoin d'un enregistrement. Par code, c'est celui passé à l'appel. Depuis le formulaire, c'est l'**Enregistrement de test** : le bouton **Exécuter l'appel** l'utilise, le tableau des variables détectées affiche les valeurs lues dessus, et **Corps pour l'enregistrement de test** montre le corps qui partirait. Il ne sert qu'aux essais.

8 Aperçu de la requête

Le bouton **Aperçu de la requête** ouvre la liste des variables détectées, chacune avec la valeur que l'appel utiliserait maintenant et sa source actuelle. Une valeur modifiée dans la liste recalcule aussitôt la méthode, l'URL, les en-têtes et le corps. Rien n'est enregistré sur le point de terminaison et rien n'est envoyé.

Une valeur vidée laisse son `{nom}` tel quel. Une erreur, par exemple une valeur obligatoire vide, s'affiche en rouge. Les en-têtes d'authentification (`Authorization`, `X-API-Key`…) sont tronqués.

9 Exécution automatique

9.1 Depuis une action planifiée

Le code d'une action planifiée ou d'une action serveur appelle `execute_call` sur le point de terminaison, en lui passant l'enregistrement à envoyer, les valeurs des variables, ou les deux.

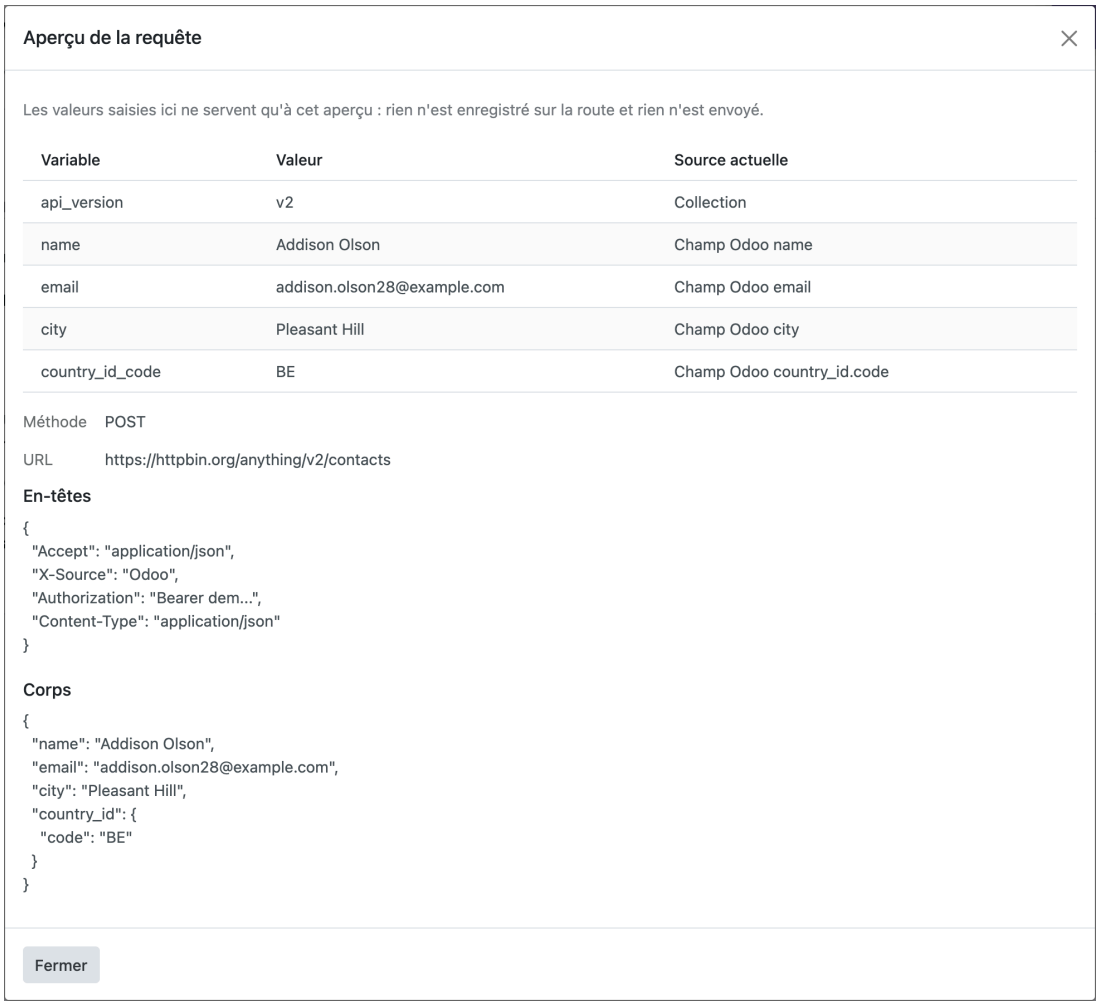


Fig. 11. – Aperçu de la requête : la valeur saisie pour le code pays passe dans l'URL et dans le corps.

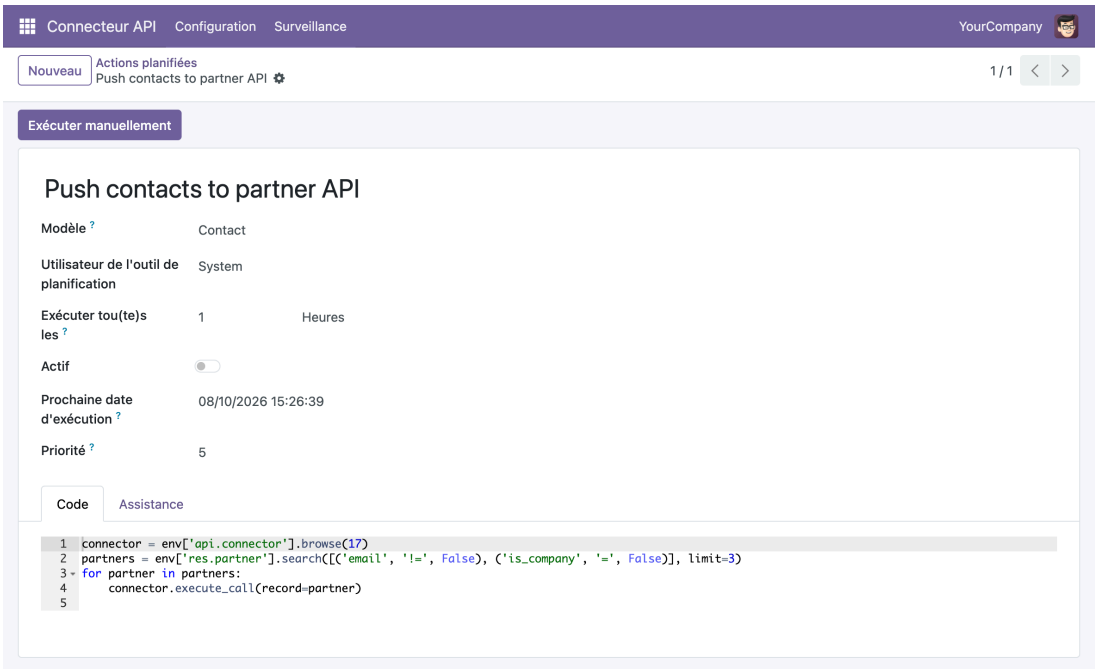


Fig. 12. – Action planifiée qui envoie trois contacts : un appel par contact, chacun avec ses propres valeurs.

```
connector = env['api.connector'].browse(17)
partners = env['res.partner'].search([('email', '!=', False)], limit=3)
for partner in partners:
    connector.execute_call(record=partner)
```

Les variables `name`, `email` et `city` du point de terminaison lisent les champs du contact. Sans modèle source, on passe les valeurs : `connector.execute_call({'name': partner.name})`. L'identifiant du point de terminaison figure dans son onglet **Documentation**.

9.2 Lire le résultat

`execute_call` renvoie `True` et ne lève pas d'erreur quand l'API répond en échec : le résultat se lit ensuite sur le point de terminaison.

Champ	Contenu
`last_response_success`	Vrai si le statut HTTP est inférieur à 400
`last_response_status`	Statut HTTP, 0 sans réponse
`last_response_data`	Données reçues, en texte (JSON indenté pour une réponse JSON)
`last_response_error`	Message d'erreur, vide en cas de succès
`last_response_time`	Durée en millisecondes

```
connector.execute_call({'name': partner.name})
if connector.last_response_success:
    data = json.loads(connector.last_response_data)
```

`execute_call` lève une erreur si le point de terminaison est archivé, ou si l'enregistrement passé n'est pas du modèle source. Un corps invalide, une valeur obligatoire vide ou un enregistrement manquant pour des variables qui lisent un champ sont enregistrés comme un échec, dans la dernière réponse et le journal, sans erreur levée.

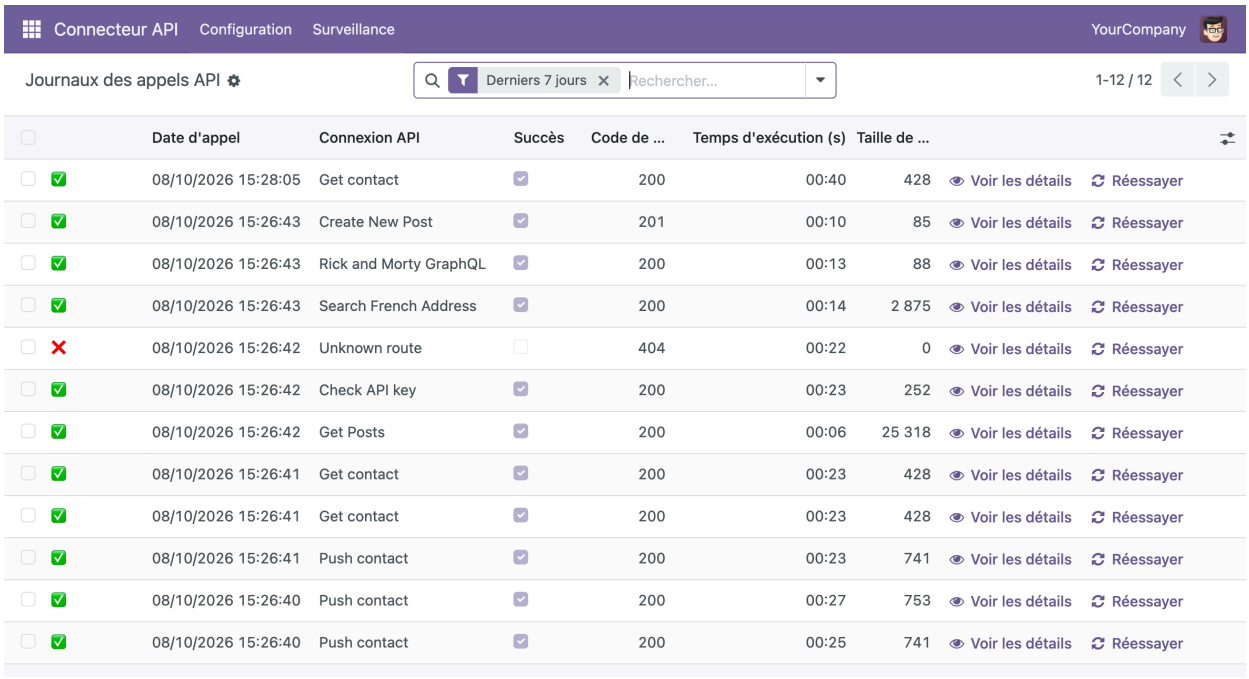


Fig. 13. – Journal des appels : la route inconnue renvoie un 404 et compte comme un échec.

10 Surveillance

10.1 Journal des appels

Surveillance > Journaux d'appel liste chaque appel : statut, date, point de terminaison, réussite, code HTTP, durée et taille de la réponse. Les filtres permettent de ne garder que les succès ou les échecs, et de regrouper par point de terminaison, statut, code ou jour.

Le détail d'un appel montre les données passées à l'exécution, les données reçues et, pour un échec, le message d'erreur.

Le bouton **Réessayer** relance l'appel avec les mêmes données d'exécution. Le nouvel appel s'ajoute au journal.

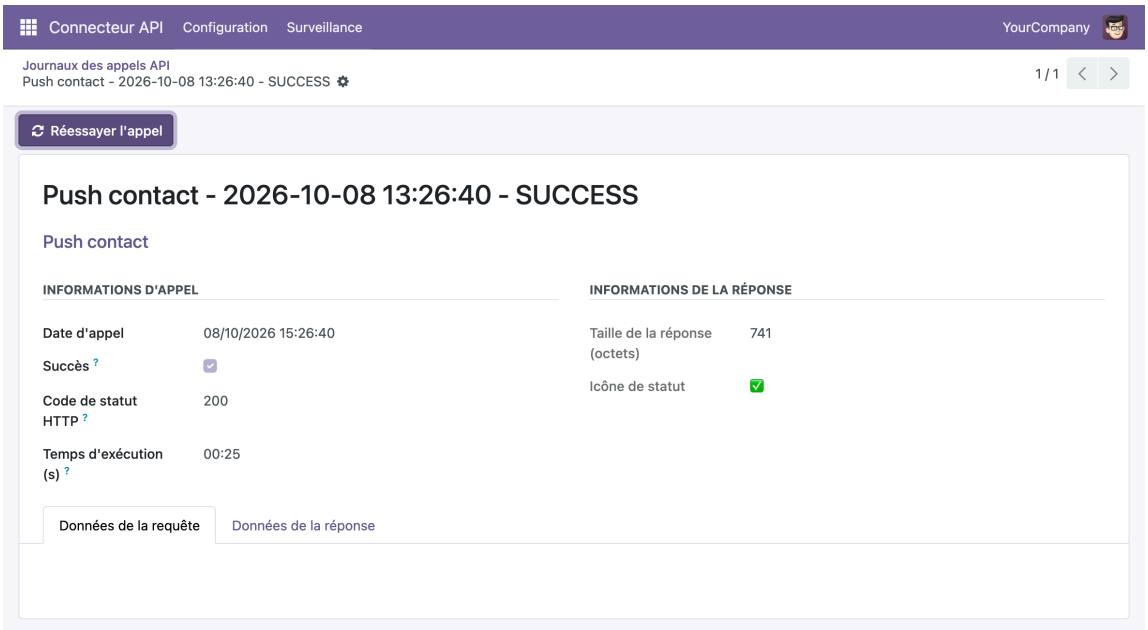


Fig. 14. – Détail d'un appel lancé par l'action planifiée : les valeurs envoyées pour ce contact.

10.2 Conservation

L'action planifiée **API Connector: Cleanup Old Logs** tourne chaque jour et supprime les entrées de plus de 30 jours. La durée se règle dans le code de l'action serveur associée (`model.cleanup_old_logs(days=30)`).

11 Export et import

11.1 Exporter

Le bouton **Configuration d'exportation** d'un point de terminaison ou d'une collection télécharge un fichier JSON. L'export d'une collection contient ses points de terminaison. Un point de terminaison exporte aussi ses variables avec leur champ Odoo, son modèle source et ses lignes de corps ; à l'import, ce modèle doit exister dans la base.

> **Attention :** le fichier exporté contient les identifiants renseignés : mots de passe, jetons, clés API et secrets OAuth. Il se conserve et se transmet comme un mot de passe.

11.2 Importer

Configuration > Importer la configuration ouvre l'assistant d'import.

Type de configuration	Effet
Détection automatique	Reconnaît le type d'après le contenu du fichier.
Connecteur API	Crée un point de terminaison, dans la **Collection cible** choisie ou, à défaut, dans la collection de même nom que celle de l'export si elle existe.
Collection API	Crée la collection et ses points de terminaison.
Postman Collection	Convertit une collection Postman.

L'import crée toujours de nouveaux enregistrements : il ne met pas à jour une configuration existante.

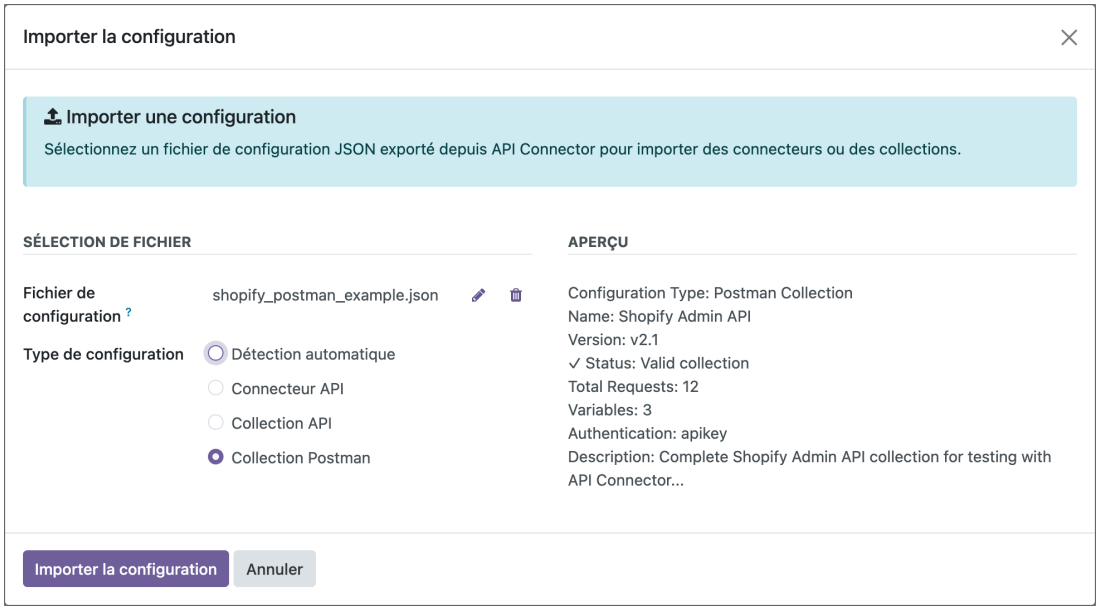


Fig. 15. – Assistant d'import : la collection Postman est reconnue et résumée avant l'import.

11.3 Collections Postman

Versions acceptées : v2.0, v2.1 et v2.2 (export **Collection v2.1** de Postman).

Élément Postman	Résultat dans Odoo
Collection	Une collection, avec ses variables et son authentification.
Dossiers	Aplatis : le nom du dossier préfixe celui de la requête (`Products - Get All Products`).
Requête	Un point de terminaison REST, avec sa méthode, son chemin et ses en-têtes.
`{{variable}}`	Converti en `{variable}` dans le chemin et le corps.
Paramètres de requête et variables de chemin	Variables du point de terminaison. Une variable de chemin reçoit la valeur provisoire `{var_nom}`.
Corps raw	JSON s'il est valide, sinon texte brut.
Corps GraphQL	Corps JSON `{"query": ..., "variables": ...}`.
Authentification basic, bearer, apikey	Reprise avec ses valeurs.

L'assistant refuse une collection qui contient des scripts (pre-request ou test), une authentification NTLM, Hawk, AWS Signature ou EdgeGrid, un corps de type fichier, des certificats ou des réglages de proxy. Une requête dont le corps est en `form-data` ou `x-www-form-urlencoded` fait échouer l'import : la retirer de la collection avant l'export, puis la créer dans Odoo.

Après l'import :

1. renseigner l'URL de base de la collection, que l'import ne remplit pas ;
2. remplacer les valeurs provisoires `{var_nom}` des variables de points de terminaison, qui l'emportent sur les variables de collection de même nom ;
3. saisir les vrais identifiants : un jeton écrit `{{variable}}` dans Postman est repris tel quel.

12 Droits d'accès

Groupe	Droits
Utilisateur interne	Lecture, création, modification et suppression des collections et des points de terminaison. Lecture du journal.
Administration (Paramètres)	Tous les droits, y compris sur le journal.

Le module crée aussi les groupes **Utilisateur de Connecteur API** et **Gestionnaire de Connecteur API**. Les droits ci-dessus ne dépendent pas de ces groupes.

Les identifiants (mots de passe, jetons, clés, secrets) sont enregistrés dans la base de données et masqués à l'écran. Tout utilisateur interne qui ouvre un point de terminaison peut l'exécuter.

13 Référence technique

13.1 Modèles

Modèle	Rôle
`api.collection`	Collection et réglages partagés

`api.connector`	Point de terminaison, exécution et dernière réponse
`api.variable`	Variable d'une collection ou d'un point de terminaison, avec son éventuel champ Odoo
`api.body.line`	Ligne d'un corps construit
`api.call.log`	Une ligne par appel
`api.request.preview`	Aperçu de la requête (modèle transitoire, avec ses lignes `api.request.preview.line`)
`import.config.wizard`	Assistant d'import
`postman.parser`	Conversion des collections Postman (modèle abstrait)

13.2 Méthodes utiles

Modèle	Méthode	Effet
`api.connector`	`execute_call(dynamic_data=None, record=None)`	Exécute l'appel avec les valeurs facultatives `dynamic_data` et l'enregistrement `record` du modèle source, journalise, met à jour les statistiques et la dernière réponse. Renvoie `True`.
`api.connector`	`add_body_field_lines(field_paths)`	Pour chaque chemin de champ (`partner_id.name`), crée la variable de route qui le lit et la ligne de corps qui l'utilise.
`api.connector`	`get_oauth_token()`	Demande un jeton OAuth 2.0 (client credentials).
`api.collection`	`get_oauth_token()`	Idem au niveau de la collection.
`api.connector`, `api.collection`	`export_config()`	Crée la pièce jointe JSON de l'export.
`api.call.log`	`cleanup_old_logs(days=30)`	Supprime les entrées plus anciennes que `days` jours.
`api.call.log`	`get_statistics(connector_id, days)`	Pour un point de terminaison (ou tous) sur `days` jours (7 par défaut), renvoie un dictionnaire : total, succès, échecs, taux de succès, durée moyenne.

13.3 Variables en JSON

Les champs `collection_variables` (collection) et `endpoint_variables` (point de terminaison) donnent les variables sous forme d'objet JSON `{nom: valeur}`. Écrire dans ces champs reconstruit les lignes de variables : un code ou un fichier de données qui les utilisait continue de fonctionner.

13.4 Supprimer un point de terminaison critique

La suppression d'un point de terminaison marqué **System Critical** est refusée. Par code, elle passe avec le contexte `force_delete_critical` : `connector.with_context(force_delete_critical=True).unlink()`.